

CASE STUDY DISCUSSION

Payment Link **Generation** Workflow

Sumanth Keshav Kotha · 23 July 2026

n8n orchestration

API calls

modular architecture

structured errors

SECTION 01

Problem & Architecture

What the workflow solves, and the shape chosen to solve it.



Collecting payment shouldn't need a workaround

- Dealerships can't generate shareable payment links inside their management system - payment collection stays manual.
- **Objective:** an orchestration layer receives request data from Tekion, generate a link, update the record.

USER STORY

As a dealership admin, I want to generate a shareable payment link directly from our system — so I can collect payment without a manual workaround, and know **for certain** whether the link was created and the record updated, even if the gateway or a downstream service is temporarily unavailable.

03 SCOPE CHECKLIST

Every scope item, built and accounted for



Receives payment request data — customer info, amount, invoice number



Calls the Payment Link API to generate a payment link



Extracts response fields

`payment_link · ID · referenceId`

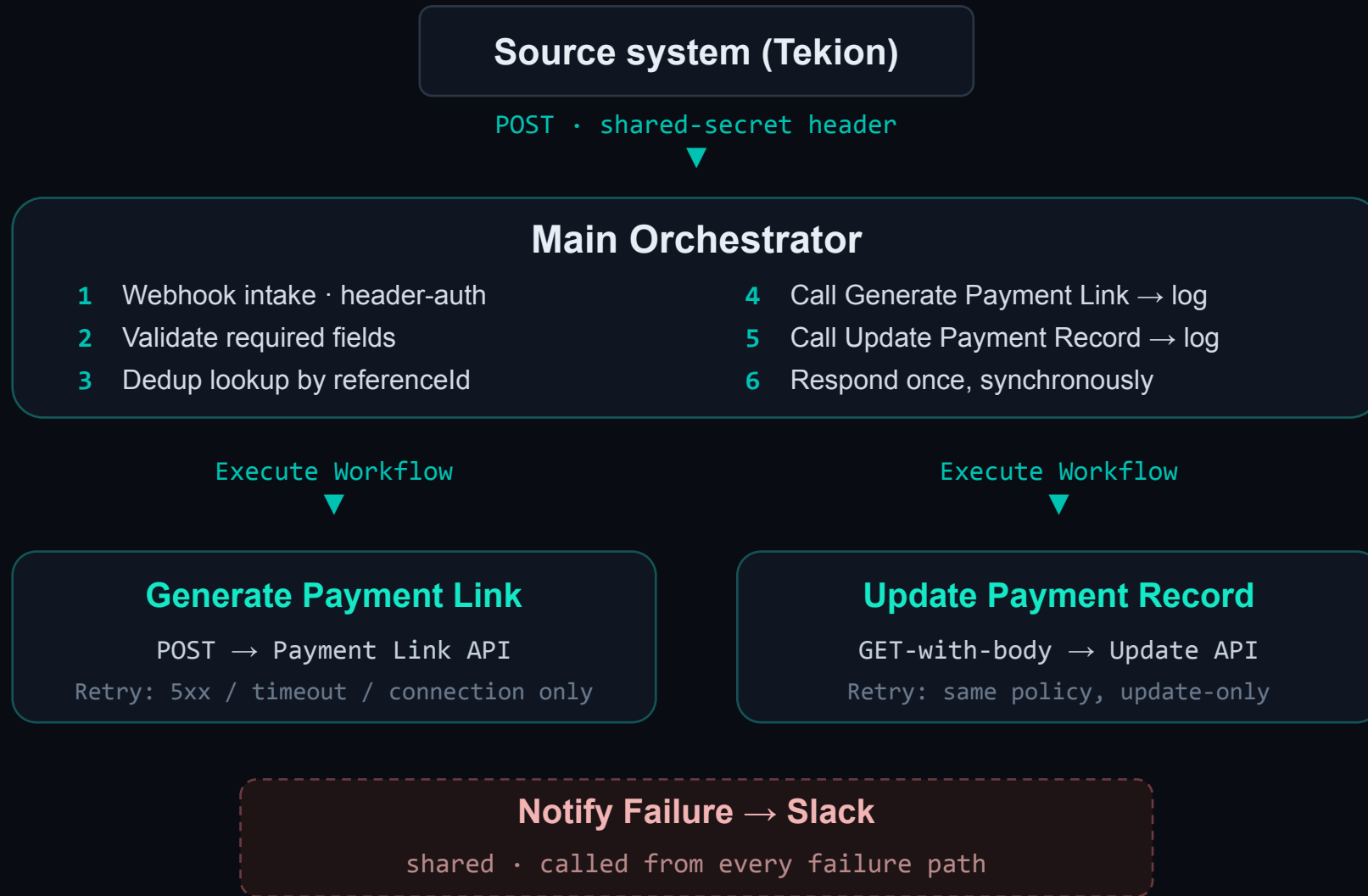


Updates the payment record via the Update API



Error handling for all failure scenarios

`proven live in the demo`



Four workflows: one orchestrator, two reusable sub-workflows, one shared failure-notifier.

Modularity you can point to, not just claim

Guideline: "The solution should be modular, maintainable, and scalable."

- **Generate** and **Update** each own their retry policy and are independently reusable - called via Execute Workflow, not inlined.
- **Notify Failure** is a single shared sub-workflow called from every failure path, not duplicated per branch.
- Change stays **contained** - swap the gateway, or add an alert channel, and only that sub-workflow changes.
- One structured envelope for every response - `{ success, referenceId, status, data:{ stage, error } }`
- same shape for a validation failure, a gateway failure, or success.

SECTION 02

Design Decisions

The choices behind retries, idempotency, assumptions, and the mock API.

Three decisions that shape behavior

- **Retry classification** - retries fire only when trying again could produce a different outcome: a server hiccup, a dropped connection, a temporary overload (up to 3 attempts). A request the gateway rejects outright is never retried - it would just fail the same way.

HTTP STATUS CODE SUMMARY

200	OK	Everything worked as expected.
400	Bad Request	The request was unacceptable, often due to missing a required parameter.
401	Unauthorized	No valid API key provided.
402	Request Failed	The parameters were valid but the request failed.
403	Forbidden	The API key doesn't have permissions to perform the request.
404	Not Found	The requested resource doesn't exist.
409	Conflict	The request conflicts with another request (perhaps due to using the same idempotent key).
424	External Dependency Failed	The request couldn't be completed due to a failure in a dependency external to Stripe.
429	Too Many Requests	Too many requests hit the API too quickly. We recommend an exponential backoff of your requests.
500, 502, 503, 504	Server Errors	Something went wrong on Stripe's end. (These are rare.)

Three decisions that shape behavior

- **Dedup key** - `referenceId`, checked against the audit Sheet before any external call, to avoid accidentally performing the same operation twice.
- **GET-with-body for the Update API** — the mock's documented contract is an unusual GET request carrying a JSON body; built to match it exactly rather than "fixing" it to a more conventional PATCH / PUT.

Assumptions made where the spec was silent

Note 1: "make reasonable assumptions where requirements are not explicitly defined and document those assumptions."

- **expireBy** - not part of Tekion's original payload; added as an optional field, defaulting to 7 days out if omitted, since the Payment Link API requires `expire_by`.
- **Webhook auth** - shared-secret header required on the inbound webhook; the assignment didn't specify an auth scheme.
- **Alerting scope** - Slack fires for validation failures, unrecoverable link / update failures, and audit-log write failures; not for routine duplicate / replay hits, since those aren't operational problems.
- **Gateway notification defaults** — *notify.email: true, reminder_enable: true*, and an empty notes object are sent as fixed values, not derived from Tekion's payload. The scope needs to be expanded if a dealership wants to own this aspect of customer communication. sample was a placeholder, so nothing meaningful was invented to fill it. All three should become caller-controlled if a dealership wants to own the customer communication.

Tolerant to the inconsistent mock response

Note 2: "The provided API is a mock implementation and may not be consistently available. Design your workflow to work with the specified request and response format."

- The Payment Link API's real response omits a couple of documented fields in practice.
- Both sub-workflows tolerate either shape and fall back to the caller's own `referenceId` when it's missing from the response.
- The Update API matches its documented shape exactly - no fallback needed there.

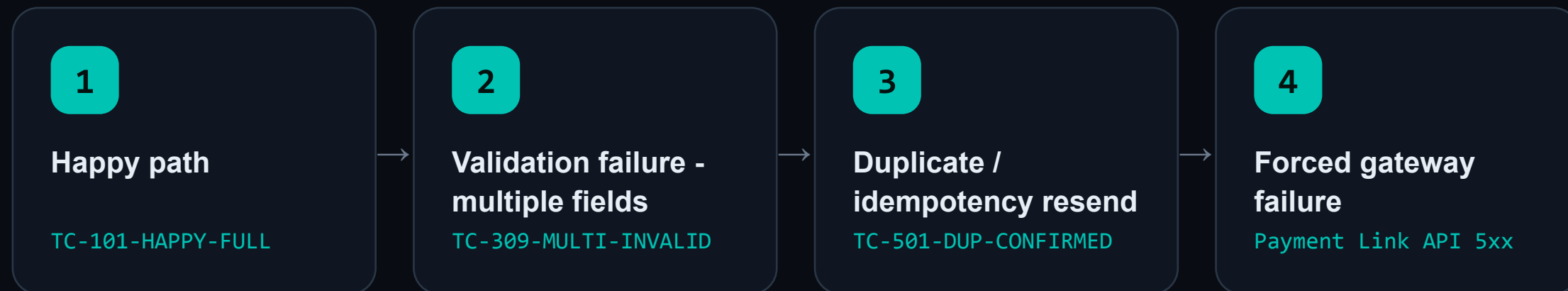
SECTION 03

Live Demo

Four cases run end-to-end, then the audit trail they leave behind.



Four live cases, in this order



10

DEMO CASE 1 — HAPPY PATH

Full payload in, link out

referenceId TC-101-HAPPY-FULL

all fields present

• Expect 200 OK

```
{
  "success": true,
  "referenceId": "TC-101-HAPPY-FULL",
  "status": "Payment link generated and record updated",
  "data": { "payment_link": "https://api.xyz.com/v1/payment_links/123456"
}
```

› switch to terminal – run the curl live

11 DEMO CASE 2 — VALIDATION FAILURE

Every failure collected, not just the first

referenceId TC-309-MULTI-INVALID

• Expect 400 Bad Request

- Payload: negative `paymentAmount`, lowercase `currency`, malformed `customerEmail`.
- `data.fields` contains **all three** failures — the validator collects every failure, not just the first.



Validation Failed

› switch to terminal — run the curl live

12

DEMO CASE 3 — DUPLICATE / IDEMPOTENCY

Second call short-circuits, no external work

referenceId TC-501-DUP-CONFIRMED

Call 1 → success

• Call 2 → 200 OK, fast

```
{
  "success": true,
  "referenceId": "TC-501-DUP-CONFIRMED",
  "status": "Already processed",
  "data": { "payment_link": "https://api.xyz.com/v1/payment_links/123456"
}
```

› switch to terminal – run both curls live

13

DEMO CASE 4 — FORCED GATEWAY FAILURE

A 5xx maps to a clean, retryable error

Point Call Payment Link API →
httpbin.org/status/503

• Expect 502 • GATEWAY_UNAVAILABLE • retryable: true

- Temporarily point **Generate Payment Link** → **Call Payment Link API** at a failing endpoint.
- **Restore the real endpoint immediately after** the run.



Payment Link Generation Failed

› switch to n8n UI – swap the URL, run the curl, then swap it back

Every phase leaves a checkpoint

- Open the Google Sheet after the four runs — phase-status columns are populated per case.
- Each column is a checkpoint: populated only once that phase is reached, blank if never attempted.

`validation_status`

Required-field checks

`link_generation_status`

Payment Link API result

`record_update_status`

Update API result

`final_status`

Overall outcome

SECTION 04

Limitations, Future & Deliverables

What this design doesn't cover yet, where it goes next, and what's shipped.

Known limitations & production considerations

- **Dedup race window** — the Sheets lookup and row write aren't atomic; two near-simultaneous requests for the same `referenceId` could both pass. Separately, three writes per request against Google's 60-writes-per-minute-per-user quota caps throughput around 20 requests/minute.
- **No async reconciliation** for process-level crashes — recovery is workflow-level (in-request retries) only.

Where this design goes next

- **Async reconciliation sweep** — a scheduled companion workflow to catch process-level crashes the in-request retries can't cover.
- **Stricter idempotency** — reject a resend carrying different field values under the same `referenceId`, instead of silently discarding them.

Every deliverable, shipped



Workflow designs, explanation & assumptions

this deck + README.md



Workflow export / source code

repo – workflows/*.json



Setup and execution instructions

README.md



Short demo video explaining the workflow

Loom walkthrough